

Take-Home Assignment

Agentic Patient Triage Workflow

LuminAI Health · AI & ML Scientist

Confidential · For Candidate Use Only

Context

Most rural households lack easy access to doctors. Many patients are more comfortable describing their health problems in local languages over phone calls than typing into apps.

Your task is to build an agent that acts as the **first line of guidance** for patients — offering home-care advice for minor ailments while safely escalating serious cases to partner doctors. This assignment simulates that frontline triage workflow, focusing on three core capabilities:

- Collecting patient symptoms through voice interaction
- Auto-classifying condition type and severity
- Providing basic advice for mild issues — and escalating serious cases with a structured summary

Objective

Build an **agentic workflow** where a Root Agent interacts with a patient to complete the following four tasks in sequence:

1	Collect Symptoms	Engage the patient in a conversational voice interface to gather symptom information.
2	Classify Condition	Categorise the patient's condition into one of three defined categories (see Classification section below).
3	Assess Severity	Determine whether the condition is Mild or Severe, and respond accordingly.
4	Summarise & Route	For severe/emergency cases only: generate a structured summary and draft a simulated email to the partner doctor.

Condition Classification

Classify every patient interaction into one of the following three categories:

Category	Examples	Action
----------	----------	--------

Upper Respiratory Issue	Cold, flu, sore throat, nasal congestion, bronchitis	Assess severity → advise or escalate
Fever-Related Condition	Viral fever, malaria, dengue, typhoid	Assess severity → advise or escalate
Other / Out-of-Scope	Symptoms that do not fit the above two categories	Route directly to doctor — no home advice

Severity Assessment

Mild	Severe / Emergency
Provide empathetic, short spoken guidance. Suggest simple, practical home remedies. Reassure the patient appropriately.	Raise a red flag and strongly advise consulting a doctor. Generate a structured case summary (see below). Route to partner doctor via email.

Technical Requirements

1. LLM Integration

Model	GroqCloud — use llama-3-70b-instruct (or equivalent Llama-3 variant available on GroqCloud)
Purpose	Natural, empathetic patient conversation and triage reasoning

2. Voice Features

ASR (Speech-to-Text)	Use any open-source or free API (e.g. Whisper, Deepgram free tier)
TTS (Text-to-Speech)	Use any open-source or free API (e.g. ElevenLabs free tier, gTTS, Coqui)

3. Triage Logic

Classification	Auto-identify condition category using the LLM
Severity	Auto-identify mild vs. severe — algorithm-driven, not hardcoded
Mild response	Short, empathetic spoken guidance with home remedies
Severe response	Red flag alert + structured summary (symptoms, duration, severity, red flags, recommended next step)

4. Post-Conversation Email

Trigger	Generate email only for severe/emergency cases
Content	Symptoms · Classification · Severity · Recommendation

Recipient	Send to yourself (you are the simulated partner doctor)
Method	Send via MCP (mailing tool) or SMTP simulation

5. Interface

Required	Simple chat interface using Gradio or Streamlit
Bonus	Expose endpoints using FastAPI

Deliverables

Submit the following three items:

- **Python Codebase** — Clean, modular Python code for the full agentic triage workflow
- **README.md** — Setup instructions, environment variables required, and how to run the application
- **Example I/O** — Demonstrated triage flows — at minimum one mild case and one severe/emergency case, showing the full conversation, classification, advice or escalation, and (for severe cases) the email output

Tech Interview

In the tech interview, you will walk through your working prototype live. Be prepared to demonstrate:

- End-to-end voice triage flow for **2–3 patient scenarios**
- How your agent classifies condition type and assesses severity
- The mild path — spoken home remedy guidance
- The severe path — red flag response and auto-generated doctor email
- Your architecture decisions — why you chose the tools and structure you did

How Your Submission Will Be Evaluated

Area	What We Look For	Weight
Agentic Architecture	Root agent with clear tool separation and orchestration logic	25%
LLM Integration	Correct GroqCloud model usage; natural, safe conversation	15%
Voice Pipeline	Working ASR + TTS in the triage loop	20%
Triage Logic	Accurate mild/severe classification; safety-first escalation	15%
Email Output	Structured, doctor-readable summary for severe cases only	10%
Interface	Clean Gradio/Streamlit UI; FastAPI endpoints are a bonus	10%
Code Quality	Readable, modular Python; clear README; working example I/O	5%

